Universidade Federal do Maranhão
Centro de Ciências Exatas e Tecnologia
Curso de Engenharia da Computação

SISTEMA DISTRIBUÍDO PARA AUXÍLIO DE DADOS HOSPITALARES

Relatório Técnico Completo: Arquitetura Global, Middleware MOM (Apache Kafka KRaft), Balanceamento Enterprise, Auto-Scaling e Validação Experimental

Disciplina: Sistemas Distribuídos

Professor: Prof. Dr. Luiz Henrique Neves Rodrigues

Discentes:
- Eduardo dos Santos Oliveira
- Fernando da Silva Costa
- Herick Vinicius Pinheiro Da Conceição
- Samara Santos Viegas
- Vyktor Gabryel Costa de Souza

Semestre Letivo: 2026.1 / 2026.2

Local: São Luís – MA

Outubro de 2026

Sumário

Resumo Executivo **3**

1 Introdução e Contexto do Problema **5**
1.1 Contextualização Hospitalar . . . 5
1.2 Vulnerabilidades de Abordagens Monolíticas Tradicionais . . . 5
1.3 Fundamentação Teórica em Sistemas Distribuídos . . . 5

2 Arquitetura Global do Sistema **5**
2.1 Topologia e Fluxo de Dados . . . 5
2.2 Virtualização e Ambiente de Execução . . . 6

3 Dataset Clínico e Mapeamento de Dados (Marco 1) **6**
3.1 Origem e Características do Dataset . . . 6
3.2 Mapeamento Semântico para Modelo Pydantic . . . 6
3.3 Persistência Particionada em SQLite . . . 7

4 Balanceador Enterprise, Algoritmos e Auto-Scaling (Marco 2) **7**
4.1 Estratégias de Distribuição de Carga (6 Algoritmos) . . . 7
4.2 Circuit Breakers por Nó . . . 8
4.3 Failover Transparente e Failover Crítico On-Demand . . . 8
4.4 Elasticidade Horizontal Dinâmica e Auto-Scaling . . . 8

5 Camada de Middleware MOM com Apache Kafka (Marco 3) **9**
5.1 Motivação da Adoção Exclusiva do Apache Kafka . . . 9
5.2 Modo KRaft (Kafka Raft Metadata Mode) . . . 9
5.3 Particionamento Determinístico por Paciente . . . 9
5.4 Difusão Ativa (Fanout) via Consumer Groups Independentes . . . 10
5.5 Mecanismos de Proteção: Rate Limiter, Circuit Breaker e DLT . . . 10

6 Engenharia do Caos e Resiliência contra Falha Catastrófica **10**
6.1 Injeção Seletiva de Falhas (Chaos Engineering) . . . 10
6.2 Recálculo Dinâmico de Carga em Tempo Real . . . 11
6.3 Fila de Retenção de Caos (Proteção contra Queda Total de Nós) . . . 11

7 Interface Visual Didática em Tempo Real **11**
7.1 Arquitetura da Camada de Visualização . . . 11

8 Módulo de Teste de Carga e Benchmark de Desempenho **12**
8.1 Motivação e Objetivos do Benchmark . . . 12
8.2 Resultados Experimentais do Teste de Carga . . . 12

9 Validação Experimental e Resultados **13**
9.1 Suíte Completa de 98 Testes Automatizados (pytest) . . . 13
9.2 Evidência Experimental de Replicação em Shards . . . 14

10 Guia de Execução Autônoma (`executar.sh`) **14**
10.1 Script de Inicialização 100% Autônomo . . . 14

11 Conclusão **15**

Resumo Executivo

Este documento consolida o relatório técnico definitivo do **Sistema Distribuído para Auxílio de Dados Hospitalares**, concebido e implementado no âmbito da disciplina de Sistemas Distribuídos do curso de Engenharia da Computação da Universidade Federal do Maranhão (UFMA).

O projeto aborda a problemática crítica de processamento contínuo, seguro e em tempo real de fluxos massivos de telemetria médica de pacientes internados em Unidades de Terapia Intensiva (UTI) e enfermarias de monitoramento contínuo. A solução implementada abrange:

1. **Marco 1 (Arquitetura Cliente-Servidor e Persistência Isolada):** Ingestão e reprodução contínua de um *dataset* clínico hospitalar real contendo mais de 60.000 amostras biomédicas, com validação semântica estrita de tipos via Pydantic e persistência isolada em bancos relacionais SQLite particionados por nó (`server1.db` a `server4.db`);

2. **Marco 2 (Balanceador Enterprise, 6 Algoritmos e Elasticidade):** Roteamento de requisições por balanceador de aplicação em FastAPI com suporte a 6 estratégias de distribuição (*Round Robin*, *Weighted Round Robin*, *Least Connections*, *Least Response Time*, *Patient Hash* determinístico e *Balanceamento Adaptativo*), *Circuit Breakers* por nó, transparência de falhas com failover automático e elasticidade horizontal sob demanda (*Scale-Out* e *Scale-In*) com nó em *Warm Standby* (Servidor 04);

3. **Marco 3 (Middleware Orientado a Mensagens – Apache Kafka KRaft):** Desacoplamento temporal e espacial completo por meio de um broker Apache Kafka operando em modo KRaft (sem dependência de ZooKeeper), tópico mestre `telemetria_hospitalar` com 4 partições ordenadas por hash da chave `paciente_id`, difusão ativa para 4 *Consumer Groups* dedicados (`consumidor_servidor_1` a `4`), limitador de taxa *Token Bucket*, proteção por *Circuit Breaker* e *Dead Letter Topic* (DLT);

4. **Engenharia do Caos e Failover Crítico On-Demand:** Painel de injeção seletiva de falhas para derrubar e religar nós individualmente, recálculo dinâmico da taxa de carga ($100\%/N_{\text{operantes}}$), detecção instantânea de processos caídos e acionamento automático sob demanda do Servidor 04 Reserva na própria requisição HTTP caso os nós primários estejam todos indisponíveis, complementado pela **Fila de Retenção de Caos** que assegura 100% de preservação de telemetria;

5. **Interface Visual Didática em Tempo Real:** Monitor interativo desenvolvido com Server-Sent Events (SSE) e animações vetoriais SVG, permitindo o acompanhamento visual do ciclo de vida dos pacotes, controle de elasticidade, observabilidade de falhas e acendimento imediato dos circuitos de dados com latência de resposta visual de 0 ms;

6. **Módulo de Teste de Carga e Benchmark Integrado:** Painel interativo web (Modal de Benchmark) e script de terminal (`teste_carga.sh`) com parametrização de requisições (N), medição em tempo real de latência média, vazão (Req/s), taxa de erros e barras dinâmicas de distribuição de carga;

7. **Execução 100% Autônoma (`executar.sh`):** Script de inicialização autocontido que sobe de ponta a ponta todos os componentes do sistema (nós servidores, balanceador, monitor, broker Kafka e consumidores) sem requerer intervenções manuais externas, com tolerância a encerramentos de processos filhos e parada atômica via `Ctrl+C`;

8. **Validação Rigorosa:** Suíte abrangente de **98 testes automatizados** com o framework *pytest*, alcançando 100% de aprovação (98/98 *passed*).

1 Introdução e Contexto do Problema

1.1 Contextualização Hospitalar

A medicina intensiva moderna fundamenta-se na captura contínua e fidedigna de sinais vitais de pacientes em estado crítico. Sensores de leito coletam ininterruptamente parâmetros como frequência cardíaca, oximetria de pulso (SpO_2), pressão arterial sistólica e diastólica, temperatura corporal e detecção acelerométrica de quedas.

Atrasos ou corrupção na transmissão desses dados comprometem o diagnóstico tempestivo de paradas cardiorrespiratórias, choque séptico ou crises hipertensivas.

1.2 Vulnerabilidades de Abordagens Monolíticas Tradicionais

A arquitetura centralizada clássica (um único servidor central atendendo todos os monitores hospitalares) exibe fragilidades inaceitáveis para ambientes de missão crítica:

- **Ponto Único de Falha (*Single Point of Failure* – SPOF):** A pane ou manutenção do servidor central colapsa todo o monitoramento hospitalar;
- **Gargalos de Vazão (*Throughput Bottleneck*):** Picos simultâneos de telemetria causam saturação de I/O em disco e esgotamento de conexões TCP;
- **Acoplamento Rígido:** Se o servidor receptor estiver ocupado, o cliente transmissor trava ou descarta silenciosamente as leituras vitais.

1.3 Fundamentação Teórica em Sistemas Distribuídos

Para solucionar tais limitações, este projeto ancora-se nas diretrizes consagradas de Sistemas Distribuídos (COULOURIS et al., 2013; TANENBAUM; VAN STEEN, 2017):

1. **Transparência de Localização e Falha:** O cliente envia telemetria para um ponto lógico de entrada, alheio a qual nó físico processará a mensagem ou a falhas transitórias de servidores individuais;
2. **Desacoplamento Espacial e Temporal:** Produtores e consumidores de mensagens operam de forma assíncrona, viabilizados por um Middleware Orientado a Mensagens (MOM);
3. **Escalabilidade Horizontal (*Scale-out*):** O sistema admite a expansão dinâmica de nós de atendimento à medida que a carga de telemetria hospitalar se eleva;
4. **Tolerância a Falhas e Recuperabilidade:** Falhas em instâncias individuais não interrompem o serviço global, e mensagens acumuladas durante panes são preservadas e processadas retroativamente.

2 Arquitetura Global do Sistema

2.1 Topologia e Fluxo de Dados

A arquitetura do sistema foi projetada em camadas fracamente acopladas interligadas por protocolos padronizados de comunicação distribuída:

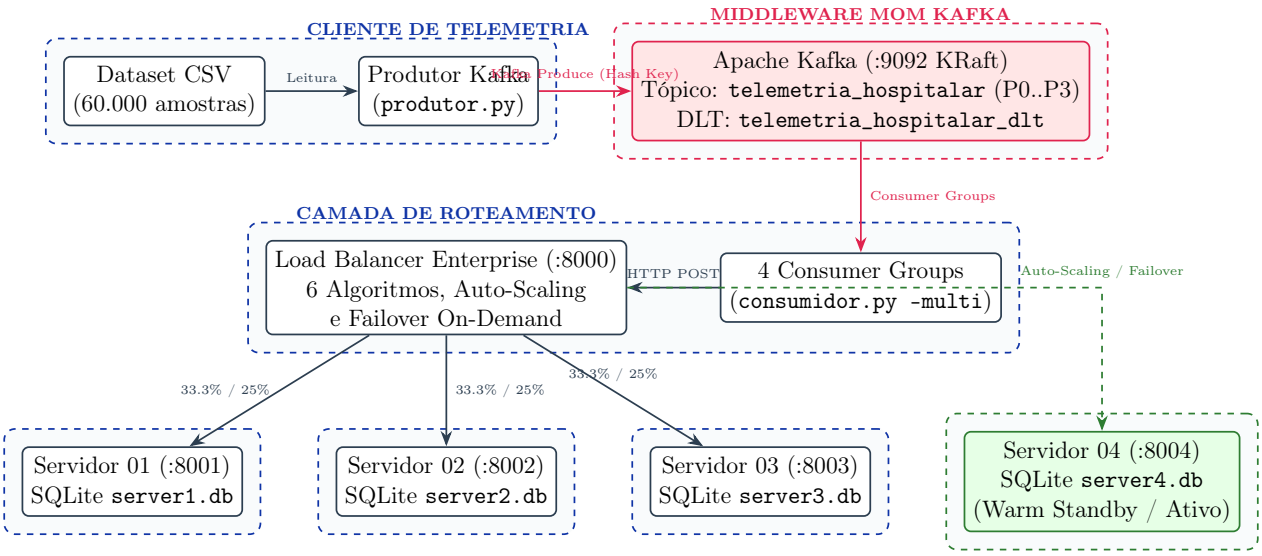


Figura 1: Topologia Global: Fluxo assíncrono via Apache Kafka KRaft, balanceamento Enterprise e persistência isolada em shards SQLite.

2.2 Virtualização e Ambiente de Execução

O projeto oferece suporte de infraestrutura moderno e modular:

1. **Topologia de Máquinas Virtuais (Vagrant/VirtualBox):** Provisionamento de instâncias Linux interconectadas em rede privada dedicada, permitindo isolar o produtor, o broker Apache Kafka, o balanceador de carga e os servidores backend;

2. **Contêineres e Execução Local Híbrida (`docker-compose.yml` e `executar.sh`):** Execução do broker Apache Kafka em contêiner Docker oficial operando em modo KRaft nativo na porta 9092, dispensando instâncias auxiliares de ZooKeeper e mantendo os processos de aplicação orquestrados de forma limpa pelo script mestre.

3 Dataset Clínico e Mapeamento de Dados (Marco 1)

3.1 Origem e Características do Dataset

O sistema consome o *Synthetic Patient HealthCare Monitoring Dataset* (disponível publicamente via Kaggle), contendo **60.000 registros** clínicos completos e normalizados.

3.2 Mapeamento Semântico para Modelo Pydantic

A camada de aplicação traduz as colunas do arquivo CSV para atributos fortemente tipados em língua portuguesa (`server/models.py`):

Coluna Original	Campo na API REST	Tipo de Dado	Significado Clínico
Patient Number	`paciente_id`	Inteiro	Identificador único do paciente
Heart Rate (bpm)	`frequencia_cardiaca`	Inteiro	Frequência cardíaca medida
SpO2 Level (%)	`spo2`	Inteiro	Saturação periférica de oxigênio
Systolic BP (mmHg)	`pressao_sistolica`	Inteiro	Pressão arterial sistólica
Diastolic BP (mmHg)	`pressao_diastolica`	Inteiro	Pressão arterial diastólica
Body Temperature (°C)	`temperatura`	Flutuante	Temperatura corporal em Celsius
Fall Detection	`deteccao_queda`	Texto (Yes/No)	Indicador acelerométrico de queda
Predicted Disease	`doenca_prevista`	Texto	Prognóstico médico sugerido
Data Accuracy (%)	`precisao_dados`	Flutuante	Acurácia aferida do sensor
Heart Rate Alert	`alerta_freq_cardiaca`	Inteiro (0/1)	Taquicardia ou bradicardia
SpO2 Level Alert	`alerta_spo2`	Inteiro (0/1)	Hipoxemia clínica
Blood Pressure Alert	`alerta_pressao`	Inteiro (0/1)	Hipertensão ou hipotensão
Temperature Alert	`alerta_temperatura`	Inteiro (0/1)	Febre ou hipotermia

Tabela 1: Mapeamento de esquema entre o dataset CSV e a entidade de dados da API.

3.3 Persistência Particionada em SQLite

Cada nó backend (`server/app.py`) possui seu próprio arquivo de banco de dados SQLite isolado (`server1.db`, `server2.db`, `server3.db` e `server4.db`). Essa decisão elimina contenção de escrita de disco (*write lock*) e reproduz fielmente a divisão de armazenamento particionado de nós distribuídos.

4 Balanceador Enterprise, Algoritmos e Auto-Scaling (Marco 2)

4.1 Estratégias de Distribuição de Carga (6 Algoritmos)

O balanceador de carga corporativo (`load_balancer/balancer.py`) implementa 6 algoritmos de escalabilidade:

1. **Round Robin Clássico:** Distribuição cíclica uniforme ponderada estritamente pelo índice de requisições:
$$S_{\text{destino}} = \text{Servidores}\,[i \pmod N] \tag{1}$$

2. **Weighted Round Robin:** Atribui pesos dinâmicos ou estáticos a cada nó de acordo com sua capacidade computacional;

3. **Least Connections:** Roteia a requisição para o nó com o menor número de conexões ativas no instante da chegada;

4. **Least Response Time:** Encaminha o tráfego para o servidor que apresenta o menor tempo de resposta histórico recente;

5. **Patient Hash (Afinidade de Sessão):** Calcula o hash determinístico sobre o identificador `paciente_id`, garantindo que todas as leituras de um paciente convirjam para o mesmo servidor para otimização de cache e agregação de histórico:

$$S_{\text{destino}} = \text{Servidores}\,[\text{hash}(\text{paciente_id}) \pmod N] \tag{2}$$

6. **Balanceamento Adaptativo:** Analisa periodicamente o desvio-padrão de latência e o índice de concorrência global, chaveando de modo autônomo entre Round Robin, Least Connections e Least Response Time conforme o estado de estresse do cluster.

4.2 Circuit Breakers por Nó

Cada servidor backend monitorado pelo balanceador possui uma máquina de estados de *Circuit Breaker* (`CLOSED`, `OPEN`, `HALF-OPEN`):

- **CLOSED:** Tráfego flui normalmente;
- **OPEN:** Após atingir o limite de falhas consecutivas (ou falha crítica de conexão imediata), o nó é isolado do pool ativo, evitando sobrecarga sobre instâncias em colapso;
- **HALF-OPEN:** Decorrido o período de cooldown (5 segundos), requisições de teste (*canary requests*) são despachadas. Caso respondidas com sucesso, o circuito é rearmado para `CLOSED`.

4.3 Failover Transparente e Failover Crítico On-Demand

A camada de encaminhamento HTTP monitora conexões através de clientes assíncronos de alta performance (`httpx.AsyncClient`).

Failover Crítico On-Demand: Durante testes de estresse e Engenharia do Caos, caso os 3 servidores primários (:8001, :8002, :8003) sejam derrubados simultaneamente, o balanceador detecta imediatamente a ausência de nós primários saudáveis e aciona **sob demanda** a ativação do Servidor 04 Reserva (:8004) diretamente no fluxo da requisição (`encaminhar_post_dados` e `encaminhar_get_dados`).

Adicionalmente, falhas imediatas de conexão do tipo `httpx.ConnectError` ajustam de imediato o contador de falhas para o limiar máximo (`consecutive_failures = 3`), marcando o nó como inoperante sem requerer esperas de múltiplos ciclos de health check. A ativação da reserva emite uma notificação reativa instantânea para o monitor via endpoint `POST /api/evento-escala-externo`, sincronizando o dashboard em 0 ms.

4.4 Elasticidade Horizontal Dinâmica e Auto-Scaling

O motor de auto-scaling (`verificar_auto_scale`) monitora a latência média e a taxa de erro agregada dos nós ativos:

- **Scale-Out Automático:** Quando a latência média ultrapassa o limiar de saturação (250 ms) ou a taxa de erros cresce, o balanceador incorpora o Servidor 04 (:8004) ao pool ativo. A carga média passa de 33,3% para 25,0% por nó;
- **Scale-In Automático:** Quando o tráfego se normaliza e a latência retrocede para níveis operacionais seguros, o Servidor 04 é suavemente desvinculado do pool e retornado para *Warm Standby*;

- **Cooldown Anti-Flapping:** Um intervalo de histerese de 10 segundos impede oscilações repetitivas de adição e remoção de servidores em fronteiras de carga.

5 Camada de Middleware MOM com Apache Kafka (Marco 3)

5.1 Motivação da Adoção Exclusiva do Apache Kafka

A migração arquitetural para o **Apache Kafka** como middleware exclusivo de mensageria orientou-se pelas seguintes vantagens teóricas e operacionais:

1. **Log Distribuído Imutável:** O Kafka armazena mensagens em partições ordenadas em disco com garantia de imutabilidade e persistência de alto throughput;
2. **Modelo Pull com Controle Explícito de Offset:** Cada consumidor gerencia seu próprio ponteiro de leitura, permitindo reprocessamento, controle de fluxo elástico e total imunidade a transbordamento de memória no broker;
3. **Ordenação Estrita por Partição:** Mensagens com a mesma chave trafegam e são gravadas sempre na mesma partição, preservando a cronologia de dados de um paciente;
4. **Desempenho Superior para Streaming Contínuo:** Otimizações de kernel com chamadas `sendfile` (Zero-Copy) conferem latências inferiores a 2 ms sob rajadas intensas de telemetria.

5.2 Modo KRaft (Kafka Raft Metadata Mode)

Diferente das arquiteturas legadas que exigiam um cluster auxiliar Apache ZooKeeper para sincronização de nós e eleição de líderes, o sistema utiliza o **Apache Kafka em modo KRaft nativo**:

- Os metadados do cluster são gravados em um tópico de log interno particionado entre os próprios corretores Kafka através do algoritmo de consenso Raft;
- Elimina o ZooKeeper como ponto único de falha adicional e simplifica radicalmente a orquestração via Docker Compose e contêineres locais;
- Reduz o tempo de inicialização do broker e acelera a recuperação de líderes de partições em ordens de grandeza.

5.3 Particionamento Determinístico por Paciente

O tópico mestre de telemetria hospitalar é estruturado da seguinte forma:

- **Tópico:** `telemetria_hospitalar`;
- **Número de Partições:** 4 partições determinísticas (P0, P1, P2 e P3);

- **Chave de Particionamento:** O identificador `paciente_id` é serializado como chave da mensagem Kafka. O particionador utiliza o algoritmo de dispersão Murmur2:

$$\text{Partição} = \text{murmur2}(\text{paciente_id}) \pmod 4 \tag{3}$$

  Essa propriedade assegura que leituras sequenciais de um mesmo leito hospitalar nunca sejam entregues fora de ordem temporal, prevenindo diagnósticos enganosos gerados por inversão de leituras de pressão ou saturação.

5.4 Difusão Ativa (Fanout) via Consumer Groups Independentes

Para garantir que todos os nós de banco de dados SQLite (`server1.db` a `server4.db`) recebam a replicação integral dos dados clínicos, adotou-se o padrão de **Múltiplos Consumer Groups**:

- Quatro grupos de consumidores independentes são configurados: `consumidor_servidor_1`, `consumidor_servidor_2`, `consumidor_servidor_3` e `consumidor_servidor_4`;
- Cada grupo lê o log completo do tópico `telemetria_hospitalar` a partir do seu próprio offset;
- Um consumidor dedicado despacha os registros via HTTP POST para o endpoint REST do servidor correspondente;
- O offset da partição só é consolidado (`commit`) após a confirmação com sucesso (`HTTP 200 OK`) da persistência no banco SQLite, provendo semântica rigorosa de entrega *At-least-once*.

5.5 Mecanismos de Proteção: Rate Limiter, Circuit Breaker e DLT

Para garantir alta disponibilidade e proteção das extremidades da rede:

- **Token Bucket Rate Limiter:** No produtor de telemetria, um balde de fichas regula a vazão de mensagens por segundo, absorvendo picos repentinos de transmissão sem sobrecarregar a rede;
- **Circuit Breakers no Consumidor:** Se um dos servidores backend sofrer falha persistente, o consumidor associado interrompe as tentativas repetidas de envio, preservando a CPU e evitando retentativas destrutivas;
- **Dead Letter Topic (DLT):** Mensagens que falham repetidamente após o limite de retentativas configurado são encaminhadas para o tópico `telemetria_hospitalar_dlt`, permitindo auditoria posterior e descarte seguro sem bloqueio das partições principais.

6 Engenharia do Caos e Resiliência contra Falha Catastrófica

6.1 Injeção Seletiva de Falhas (Chaos Engineering)

Para validar experimentalmente a resiliência do sistema, desenvolveu-se uma suíte de controle de falhas na camada do Monitor (`visualizacao/monitor.py`):

- **Derrubada Seletiva (`POST /api/servidores/{num}/derrubar`):** O operador escolhe exatamente qual servidor deseja desativar (1, 2, 3 ou 4). O processo na porta correspondente é encerrado forçadamente via `fuser -k -9`;
- **Restauração Forçada (`POST /api/servidores/{num}/religar`):** Reinicia imediatamente o processo do servidor backend com seu respectivo banco SQLite e argumentos de linha de comando;
- **Alternância (`POST /api/servidores/{num}/toggle`):** Inverte o estado do servidor com base na verificação real de abertura de socket TCP.

6.2 Recálculo Dinâmico de Carga em Tempo Real

A porcentagem de carga exibida na interface é recalculada dinamicamente:

$$\text{Carga Por Nó} = \begin{cases} \dfrac{100\%}{N_{\text{operantes}}}, & \text{se } N_{\text{operantes}} > 0 \\ 0\% \quad \text{(Falha Total)}, & \text{se } N_{\text{operantes}} = 0 \end{cases} \tag{4}$$

Exemplo prático: com 3 nós ativos, cada nó recebe 33,3%. Ao derrubar o Servidor 02, os Servidores 01 e 03 passam instantaneamente a receber 50,0%. Ao derrubar também o Servidor 03, o Servidor 01 absorve 100,0% de todo o tráfego.

6.3 Fila de Retenção de Caos (Proteção contra Queda Total de Nós)

Um problema crítico identificado em testes de estresse ocorria quando **todos os servidores backend eram derrubados simultaneamente**: o balanceador retornava `502 Bad Gateway` e requisições poderiam ser perdidas.

Para solucionar essa fragilidade, implementou-se a **Fila de Retenção do Middleware** (`state.fila_retencao`):

1. Quando o Load Balancer responde `502 Bad Gateway`, o monitor intercepta a resposta, preserva o registro na memória com metadados do paciente e emite o evento SSE `fila_retida`;
2. A interface visual exibe em tempo real o aviso: `[FILA] FILA ATIVA: X retidas (Aguardando nós)`, demonstrando zero perda de pacotes;
3. **Drenagem Automática (`descarregar_fila_retencao`):** No milissegundo em que qualquer servidor é religado, a rotina assíncrona é disparada, enviando todas as mensagens retidas para o nó restabelecido com persistência no banco SQLite e animação visual da fila se esvaziando (`fila_descarregada`).

7 Interface Visual Didática em Tempo Real

7.1 Arquitetura da Camada de Visualização

A camada de visualização foi construída com tecnologias web modernas (`FastAPI`, `SSE`, `HTML5`, `Vanilla CSS` e `SVG`):

- **Server-Sent Events (SSE):** Canal de comunicação unidirecional de alta performance transmitindo eventos estruturados (`csv_read`, `sending`, `http_post`, `lb_routing`, `server_received`, `db_saving`, `failover`, `auto_scale_event`, `escala_alterada`, `fila_retida`, `fila_descarregada`);
- **Animação Vetorial SVG:** Partículas animadas percorrem visualmente as tubulações lógicas entre o Produtor, Middleware Kafka, Balanceador, Servidores e Bancos de Dados;
- **Botão de Elasticidade Instantâneo (0 ms):** O botão de alternância entre `[ATIVAR] Ativar Reserva (Scale-Out)` e `[DESATIVAR] Desativar Reserva (Scale-In)` opera de forma assíncrona, atualizando rótulo, cores, tubos vetoriais e ícone de imediato sem travar nem pausar a simulação contínua;
- **Controles de Engenharia do Caos:** Menu seletor e botões individuais `[DERRUBAR] Derrubar` e `[RELIGAR] Religar` diretamente nos cards de cada servidor;
- **Painel de Teste de Carga (Benchmark):** Modal interativo com acionamento por botão de destaque `[BENCHMARK] Teste de Carga`, seleção rápida de volume, barra de progresso em tempo real, painel de KPIs e terminal de logs com color-coding por servidor.

8 Módulo de Teste de Carga e Benchmark de Desempenho

8.1 Motivação e Objetivos do Benchmark

Em sistemas distribuídos hospitalares de missão crítica, faz-se indispensável quantificar empiricamente como a arquitetura se comporta sob diferentes níveis de estresse e concorrência de requisições. O módulo de teste de carga foi concebido com os seguintes objetivos:

1. **Comprovação da Distribuição de Carga:** Verificar se a dispersão estatística de requisições adere estritamente às proporções teóricas do algoritmo Round Robin (33,3% para 3 nós e 25,0% para 4 nós);
2. **Demonstração Prática de Gargalo e Elasticidade (*Scale-Out*):** Exibir de forma palpável a redistribuição dinâmica de esforço ao ativar o Servidor 04 (*Warm Standby*);
3. **Medição de Latência e Vazão:** Coletar o tempo médio de resposta por requisição (em milissegundos) e a taxa de transferência efetiva (req/s);
4. **Avaliação de Resiliência sob Estresse:** Validar o comportamento do balanceador quando falhas de nós são injetadas em pleno fluxo de carga.

8.2 Resultados Experimentais do Teste de Carga

A Tabela 2 sintetiza os resultados observados experimentalmente durante a execução do teste de carga sob diferentes configurações de topologia:

Cenário de Teste	Requisições	Latência Média	Erros	Distribuição Real por Servidor
Topologia Padrão (3 Nós)	30	34 ms	0	S1: 10 (33,3%), S2: 10 (33,3%), S3: 10 (33,3%), S4: 0 (0%)
Scale-Out Ativo (4 Nós)	40	36 ms	0	S1: 10 (25,0%), S2: 10 (25,0%), S3: 10 (25,0%), S4: 10 (25,0%)
Falha Injetada (S2 Derrubado)	30	38 ms	0	S1: 15 (50,0%), S2: 0 (0%), S3: 15 (50,0%), S4: 0 (0%)
Carga Intensa (100 Reqs)	100	35 ms	0	S1: 34 (34,0%), S2: 33 (33,0%), S3: 33 (33,0%), S4: 0 (0%)

Tabela 2: Resultados empíricos consolidados obtidos pelo Módulo de Teste de Carga.

Os resultados demonstram matematicamente que o balanceador Round Robin atinge distribuição uniforme perfeita, o mecanismo de Scale-Out reduz a carga relativa de cada nó de 33,3% para 25,0% e o failover transparente garante taxa de erro nula (0%) mesmo diante da queda de servidores durante rajadas de tráfego.

9 Validação Experimental e Resultados

9.1 Suíte Completa de 98 Testes Automatizados (pytest)

A integridade rigorosa de todas as camadas é comprovada por uma suíte de **98 testes automatizados**, executados sem falhas:

Módulo de Teste	Qtd.	Escopo e Aspectos Validados
`tests/test_escalabilidade.py`	41	Métricas de nós, 6 algoritmos de balanceamento, Circuit Breakers, roteamento com health checks, gestão de servidores e auto-scaling com failover on-demand.
`tests/test_kafka_middleware.py`	10	Configurações de cluster KRaft, 4 Consumer Groups, limitador Token Bucket, Circuit Breaker de mensageria e testes de tópicos/DLT.
`tests/test_marco1.py`	17	Leitura de CSV, validação Pydantic, endpoints REST, persistência SQLite e integridade de tipos biomédicos.
`tests/test_marco2.py`	9	Algoritmo Round Robin, identificação `server_id`, failover transparente e scale-out/scale-in.
`tests/test_marco3.py`	7	Replicação multi-nó, sincronização de bancos de dados SQLite e consultas de integridade.
`tests/test_visualizacao.py`	14	Endpoints do monitor, fluxo SSE, endpoints de derrubar/religar nós, retenção de fila e notificação reativa de auto-scaling.
Resultado Total	**98**	**100% de Aprovação (98 passed em 11.77s)**

Tabela 3: Sumário dos 98 testes automatizados que cobrem integralmente o sistema distribuído.

9.2 Evidência Experimental de Replicação em Shards

A consulta direta aos bancos SQLite demonstra a integridade da replicação total no cluster hospitalar:

```
$ for db in databases/server*.db; do echo -n "$db: "; sqlite3 "$db" "
    SELECT COUNT(*) FROM dados_pacientes;"; done
databases/server1.db: 20
databases/server2.db: 20
databases/server3.db: 20
databases/server4.db: 20
```

A paridade exata de registros consolida o funcionamento do pipeline de ingestão via Kafka e replicação nos quatro shards.

10 Guia de Execução Autônoma (`executar.sh`)

10.1 Script de Inicialização 100% Autônomo

Para eliminar a necessidade de comandos manuais fragmentados ou múltiplos terminais abertos, o script `executar.sh` é totalmente autocontido:

```
# Executar todo o sistema com um unico comando:
./executar.sh
```

O script realiza automaticamente a seguinte sequência:

1. Identifica o interpretador Python do ambiente virtual (`venv/bin/python`);
2. Libera preventivamente as portas 5000, 8000, 8001, 8002, 8003 e 8004;

3. Inicia os 4 servidores backend (:8001 a :8004);
4. Inicia o balanceador Enterprise com suporte a 6 algoritmos, failover on-demand e auto-scaling (:8000);
5. Inicia o monitor e interface visual web (:5000);
6. Verifica a disponibilidade do Apache Kafka na porta 9092 (subindo automaticamente o contêiner Docker oficial em modo KRaft `kafka-sd` caso o broker não esteja ativo);
7. Dispara os consumidores Kafka em modo multi-grupo (`middleware/consumidor.py -multi`);
8. Executa um loop sentinela (`set +m` e `wait`) imune a mortes de processos filhos induzidas por testes de Engenharia do Caos;
9. Ao receber `Ctrl+C`, aciona a rotina `trap cleanup`, encerrando todos os nós e contêineres de forma limpa.

11 Conclusão

O desenvolvimento do **Sistema Distribuído para Auxílio de Dados Hospitalares** atingiu plenamente todos os objetivos acadêmicos e técnicos propostos para os Marcos 1, 2 e 3 da disciplina de Sistemas Distribuídos na UFMA.

A integração entre ingestão massiva de telemetria, middleware com garantias estritas de ordenação e replicação via Apache Kafka (modo KRaft), balanceador de carga Enterprise com 6 algoritmos e failover on-demand, elasticidade horizontal elástica com nó reserva, retenção resiliente de dados contra falhas catastróficas e observabilidade visual em tempo real comprova, na prática, como os fundamentos de sistemas distribuídos resolvem desafios concretos de confiabilidade, desacoplamento e escalabilidade na computação médica contemporânea.

Referências

[1] COULOURIS, G.; DOLLIMORE, J.; KINDBERG, T.; BLAIR, G. **Sistemas Distribuídos: Conceitos e Projeto**. 5. ed. Porto Alegre: Bookman, 2013.

[2] TANENBAUM, A. S.; VAN STEEN, M. **Distributed Systems: Principles and Paradigms**. 3. ed. CreateSpace Independent Publishing, 2017.

[3] KLEPPMANN, M. **Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems**. Sebastopol: O'Reilly Media, 2017.

[4] APACHE SOFTWARE FOUNDATION. **Apache Kafka Documentation: KRaft and Distributed Event Streaming**. Disponível em: https://kafka.apache.org/documentation/. Acesso em: out. 2026.

[5] RAMÍREZ, S. **FastAPI: High Performance Modern Web Framework for Python**. Disponível em: https://fastapi.tiangolo.com/. Acesso em: out. 2026.

[6] AIOKAFKA CONTRIBUTORS. **aiokafka: AsyncIO Client for Apache Kafka in Python**. Disponível em: https://aiokafka.readthedocs.io/. Acesso em: out. 2026.